DOCKER — EXERCICIS DE CREACIÓ D'IMATGES

SOLUCIONARI

En totes les solucions, `<usuari>` és el nom d'usuari de Docker Hub de l'alumne. Cada ordre s'executa dins de la carpeta de l'exercici (`~/docker-imatges/exN`), que conté el `Dockerfile` i l'`index.html`.

Exercici 1

Dockerfile:

```
FROM nginx:1.27-alpine
RUN apk add --no-cache nano
COPY index.html /usr/share/nginx/html/index.html
```

Ordres:

```
docker build -t <usuari>/web-nginx:1.0 .
docker run -d --name web1 -p 8081:80 <usuari>/web-nginx:1.0
curl http://localhost:8081
docker exec web1 nano --version
```

Notes per a la correcció:

• El port intern sempre és el 80, tant en NGINX com en Apache. Un error típic és posar `-p 8081:8081`.

• En les imatges Alpine s'usa `apk add --no-cache` (sense `update` previ). En les Debian cal `apt-get update` abans de l'`install` i `-y` perquè no es quede esperant confirmació.

• Netejar `/var/lib/apt/lists` no és obligatori, però redueix la mida de la imatge. És bona pràctica fer-ho dins del mateix `RUN`.

• També és vàlid copiar a la carpeta sense posar el nom del fitxer de destí (`COPY index.html /usr/share/nginx/html/`).

Exercici 2

Dockerfile:

```
FROM httpd:2.4.62
RUN apt-get update && \
    apt-get install -y curl && \
    rm -rf /var/lib/apt/lists/*
COPY index.html /usr/local/apache2/htdocs/index.html
```

Ordres:

```
docker build -t <usuari>/web-apache:1.0 .
docker run -d --name web2 -p 8082:80 <usuari>/web-apache:1.0
curl http://localhost:8082
docker exec web2 curl --version
```

Exercici 3

Dockerfile:

```
FROM nginx:1.26
RUN apt-get update && \
    apt-get install -y nano && \
    rm -rf /var/lib/apt/lists/*
COPY index.html /usr/share/nginx/html/index.html
```

Ordres:

```
docker build -t <usuari>/web-nginx:2.0 .
docker run -d --name web3 -p 8083:80 <usuari>/web-nginx:2.0
curl http://localhost:8083
docker exec web3 nano --version
```

Exercici 4

Dockerfile:

```
FROM httpd:2.4-alpine
RUN apk add --no-cache curl
COPY index.html /usr/local/apache2/htdocs/index.html
```

Ordres:

```
docker build -t <usuari>/web-apache:2.0 .
docker run -d --name web4 -p 8084:80 <usuari>/web-apache:2.0
curl http://localhost:8084
docker exec web4 curl --version
```

Exercici 5

```
docker images
```

Les imatges més menudes són les basades en Alpine (`web-nginx:1.0` i `web-apache:2.0`), que ocupen unes poques desenes de MB. Les basades en Debian (`web-apache:1.0` i `web-nginx:2.0`) ocupen al voltant de 150-200 MB. La mida exacta pot variar segons la versió descarregada.

Resposta esperada: Alpine és una distribució Linux mínima pensada per a contenidors. Usa la llibreria musl en lloc de glibc i BusyBox en lloc de les eines GNU, i només porta el mínim imprescindible. Debian inclou moltes més biblioteques i utilitats. A més, `apt-get` afig la llista de paquets si no s'esborra.

Exercici 6

```
docker login
docker push <usuari>/web-nginx:1.0
docker push <usuari>/web-apache:1.0
docker push <usuari>/web-nginx:2.0
docker push <usuari>/web-apache:2.0
```

Si una imatge s'ha creat sense el nom d'usuari, el `push` falla amb `denied: requested access to the resource is denied`. Es resol reetiquetant-la, per exemple `docker tag web-nginx:1.0 <usuari>/web-nginx:1.0`. La comprovació es fa a `https://hub.docker.com/u/<usuari>`.

Exercici 7

Estructura del repositori:

```
practica-dockerfile-1/
├── README.md
├── ex1/
│   ├── Dockerfile
│   └── index.html
├── ex2/  (ídem)
├── ex3/  (ídem)
└── ex4/  (ídem)
```

Ordres (després de crear el repositori buit a GitHub):

```
cd ~/docker-imatges
git init
git add .
git commit -m "Pràctica Dockerfile 1"
git branch -M main
git remote add origin \
  https://github.com/<usuari-github>/practica-dockerfile-1.git
git push -u origin main
```

Si tenen la CLI de GitHub, poden crear-lo i pujar-lo d'una vegada amb `gh repo create practica-dockerfile-1 --public --source=. --push`. El `README.md` ha de contindre el nom de l'alumne i la resposta de l'exercici 5.